

2026 年全国大学生计算机系统能力大赛

操作系统设计赛 · OS 功能挑战赛道

面向小型操作系统的
分析比对智能体系统

OS 内核代码分析智能体（oskernel-agent）

Design and Implementation of an Agent that Describes and
Compares the Education-oriented Operating System Kernel

功能挑战赛道 · 设计方案与技术文档

赛 道	操作系统设计赛 · OS 功能挑战赛道（全国）
赛题名称	面向小型操作系统的分析比对智能体系统设计
项目编号 / 难度	project3136859-379280；难度等级 A（学术型）
命题单位	华中科技大学（命题人：邵志远）
项目名称	面向小型操作系统的分析比对智能体系统（oskernel-agent）
代码仓库	https://gitlab.eduxiji.net/T202610336999834/project3136859-379280
团队名称	海底小纵队
团队成员	牛一强、王佳鑫、王一凡
指导教师	曾英佩
学校 / 单位	杭州电子科技大学
文档版本	V1.0（功能挑战赛道设计方案与技术文档）
完成日期	2026 年 6 月

摘 要

本文面向 2026 年全国大学生计算机系统能力大赛操作系统设计赛 OS 功能挑战赛道赛题“面向小型操作系统的分析比对智能体系统设计”，设计并实现了一套面向教学型小型操作系统内核源码的智能分析与比对系统。系统以大语言模型（Large Language Model, LLM）智能体（Agent）为核心，围绕同一份内核仓库提供三条相互独立又统一服务于评审的分析链路：作品描述报告链路基于多语言符号解析与模型上下文协议（Model Context Protocol, MCP）工具集，按自底向上方式生成结构化的内核理解报告；历史作品查重链路采用 L0 文件指纹、L1 SimHash 粗筛、L2 向量召回、L3 分段验证、L4 精确比对、L5 LLM 复核的多层漏斗，并辅以元数据信号与基线公共代码扣除；AI 生成代码检测链路以免训练的 log-rank 与归一化扰动对数秩（Normalized Perturbed log-Rank, NPR）两阶段信号进行打分。针对赛题“尽量避免大模型幻觉、比较文档精准无误”的评审要点，系统引入报告行号回源后置校验、MCP 幻觉扩展守卫与探索步数预算等抗幻觉工程机制。在合成的四类已知克隆（T1-T4）与负样本上的评测中，查重链路当前版本取得查准率 1.0、查全率 0.975 的结果（数据来源：仓库内置评测记录），并通过批处理调优将整库嵌入耗时由约 187 秒降至约 6 秒。本文按功能挑战赛道提交规范，依次给出目标描述、赛题分析、系统框架设计、开发计划、重要进展、系统测试、问题与解决、分工协作、仓库目录与非本团队来源说明、比赛收获等内容，所有技术与数据结论均以项目源代码与仓库内评测记录为依据。

关键词：操作系统内核；大语言模型智能体；代码查重；语义相似度；AI 生成代码检测；模型上下文协议（MCP）；抗幻觉

目 录

第一章	目标描述	1
1.1	项目背景与面向场景.....	1
1.2	核心问题与用户对象.....	1
1.3	系统建设目标与预期功能.....	1
1.4	预期效果与赛题任务的对应关系.....	2
第二章	比赛题目分析和相关资料调研	3
2.1	赛题要求与任务边界.....	3
2.2	功能需求与技术难点.....	3
2.3	评价重点	3
2.4	同类项目与技术路线对比.....	3
2.5	本项目的优势、不足与创新点（概述）	4
第三章	系统框架设计.....	5
3.1	总体架构与设计原则.....	5
3.2	技术栈与运行环境.....	6
3.3	模块划分与共享数据底座.....	6
3.4	作品描述报告链路.....	7
3.4.1	多语言符号解析引擎.....	8
3.4.2	MCP 工具集与自底向上理解	8
3.5	历史作品查重链路.....	9
3.5.1	历史库构建.....	9
3.5.2	L1 SimHash 粗筛.....	10
3.5.3	L2 向量召回	10
3.5.4	L3 分段验证与 L4 精确比对	11
3.5.5	元数据信号与基线扣除.....	11
3.5.6	L5 LLM 复核与报告	12
3.6	AI 生成代码检测链路	13
3.7	接口设计、存储与运行流程.....	13
3.8	异常处理、降级与质量属性设计	13
第四章	开发计划	15
4.1	阶段划分与时间进度.....	15
4.2	各阶段目标与阶段性成果.....	15
4.3	后续优化计划.....	15

第五章 比赛过程中的重要进展.....	16
5.1 核心模块的迭代过程.....	16
5.2 代码结构与工程的重要变化.....	16
5.3 模型、接口与部署方面的进展.....	16
5.4 创新性分析.....	16
5.5 进展汇报幻灯片应体现的阶段性成果.....	17
第六章 系统测试情况.....	18
6.1 测试环境	18
6.2 功能测试	18
6.3 查重链路评测：方法与结果.....	19
6.4 性能与精度的进展对比.....	19
6.5 接口、兼容性与异常测试.....	20
6.6 功能完成情况.....	20
6.7 功能分析	21
6.8 性能分析	21
6.9 测试结论	21
第七章 遇到的主要问题和解决方法.....	22
7.1 环境配置与依赖冲突.....	22
7.1.1 第三方解析器依赖与本仓库 tree-sitter 版本冲突.....	22
7.1.2 Windows 环境下历史仓库检出失败	22
7.2 模型加载与推理.....	22
7.2.1 AI 检测阈值与打分模型失配.....	22
7.2.2 语义嵌入吞吐瓶颈.....	22
7.3 运行稳定性.....	23
7.3.1 智能体探索失控与幻觉扩展.....	23
7.3.2 报告渲染与数据编码崩溃.....	23
7.4 算法精度与数据处理.....	23
7.4.1 SimHash 对改名与结构重写召回不足	23
7.4.2 同源公共代码导致误报.....	23
7.5 数据可信与抗幻觉.....	24
7.5.1 LLM 生成的行号引用不可靠	24
第八章 分工和协作	25
8.1 团队分工	25
8.2 协作方式与任务推进.....	25

8.3 代码托管与版本管理.....	25
第九章 提交仓库目录和文件描述.....	26
9.1 仓库总体结构.....	26
9.2 非本团队来源说明.....	26
第十章 比赛收获	28
10.1 技术能力	28
10.2 工程实践	28
10.3 团队协作与项目管理.....	28
10.4 问题解决与文档/测试规范.....	28
参考资料	29

第一章 目标描述

1.1 项目背景与面向场景

自 2021 年全国大学生计算机系统能力大赛操作系统设计赛内核实现赛道开展以来，参赛队伍基于 RISC-V 等体系结构开发了大量设计风格各异的小型操作系统内核，并以开源代码仓库的形式提交至全国赛指定托管平台。随着比赛进入第六年，历史作品数量持续累积，组委会与评审专家在两类工作上面临日益增长的人工负担：一是对新提交作品的内核结构、调用逻辑与开发历史进行理解并形成对人类友好的描述；二是将新作品与历届存量作品进行比对，识别其与历史设计之间的技术差异与潜在抄袭，并对其中由 AI 辅助生成的代码予以甄别。

本项目即面向上述评审场景，构建一套面向教学型小型操作系统内核源码的智能分析与比对系统。系统以大语言模型（LLM）与智能体（Agent）技术为核心，超越简单文本匹配，从内核子系统结构、函数调用关系与版本演进等维度自动生成结构化描述，并实现新作品与历史作品的深度比对。

1.2 核心问题与用户对象

系统需要解决的核心问题包括三方面：其一，如何在不要人工逐文件阅读的前提下，对一份规模可观的内核仓库自动产出准确、可核查的结构化描述；其二，如何在改名、增删语句、结构重写等常见变形下仍能稳定地从历史库中召回相似实现，并控制误报；其三，如何对疑似 AI 生成的代码给出可解释的判别信号。系统的直接用户对象为大赛组委会与评审专家，潜在用户还包括内核课程的教师与助教、以及希望自查原创性的参赛队伍。

1.3 系统建设目标与预期功能

系统的总体建设目标是：围绕同一份内核仓库，提供描述、查重、AI 生成代码检测三条相互独立又统一服务于评审的分析链路，并输出对人类友好、尽量避免模型幻觉的文档。围绕该目标，系统规划的预期功能如下：作品描述报告功能，基于多语言符号解析与 MCP 工具集，按自底向上方式逐子系统理解内核并产出分层的 HTML 描述报告；历史作品查重功能，以多层漏斗在历史库中检索相似函数，生成语义查重对比报告；AI 生成代码检测功能，以训练方法对函数级代码给出原创性打分并并入查重报告。三条链路的总体关系如图 1-1 所示。

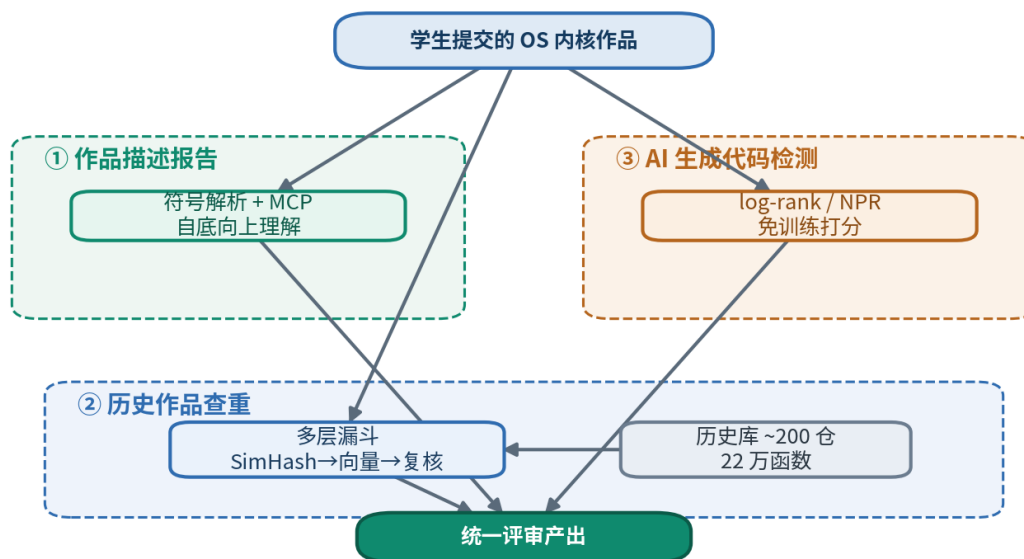


图 1-1 三条分析链路总览：同一仓库、各自独立、统一服务于评审

1.4 预期效果与赛题任务的对应关系

赛题任务要求“设计智能体对历史内核作品进行描述；将新作品与历史作品进行比较；描述与比较均产出对人类友好的文档”，并在评审要点中强调“描述文档尽量避免大模型幻觉、比较文档应做到精准无误”。本项目的预期效果与上述任务逐项对应：描述报告链路对应“描述”任务，查重链路对应“比较”任务，二者均产出 HTML 文档；针对“避免幻觉、精准无误”的要点，系统在描述链路引入行号回源后置校验与 MCP 幻觉守卫，在查重链路引入精确行级比对与 LLM 多票复核，以提升结论的可核查性。需要说明的是，本文所述功能区分“已实现”“部分实现”“后续计划”三种状态，具体完成情况以第六章功能完成情况表为准。

第二章 比赛题目分析和相关资料调研

2.1 赛题要求与任务边界

根据赛题说明，本赛题属操作系统设计赛 OS 功能挑战赛道，赛题名称为“面向小型操作系统的分析比对智能体系统设计”，难度等级 A、类型为学术型，由华中科技大学命题。赛题特征为“大模型智能体开发”，参考资料明确建议优先选用国产、开源、免费模型，并鼓励采用 MCP、检索增强生成（Retrieval-Augmented Generation, RAG）、技能（Skills）等智能体设计技术。据此，本项目的任务边界界定为：以教学型小型操作系统内核（C 与 Rust 为主）源码为分析对象，产出面向人类的描述文档与比较文档，不涉及对内核本身的功能开发或性能竞速，重点在分析与比对的准确性与可核查性。

2.2 功能需求与技术难点

从赛题任务可拆解出三项主要功能需求：内核源码的结构化描述、新旧作品的相似性比对、AI 生成代码的甄别。其对应的技术难点在于：内核仓库规模较大且跨语言，需要稳定的符号解析与按需读取机制以控制模型上下文规模；相似实现常经历改名、增删语句乃至控制结构重写，纯文本或词法级比对难以稳定召回；大模型在长上下文分析中易产生幻觉，描述与比较结论必须可回溯到真实代码行；国产、开源、免费模型的选型与成本控制需与工程实现相协调。

2.3 评价重点

结合赛题评审要点，本项目将评价重点理解为三条：一是描述文档对人类的友好程度与抗幻觉能力，即结论是否准确、是否可核查；二是比较文档的精准程度，即在保证召回的同时尽量降低误报；三是系统作为大模型智能体在工程实现上的完整性，包括工具协议、检索增强、技能体系与成本控制。后续章节的设计与测试均围绕上述重点展开。

2.4 同类项目与技术路线对比

为厘清本项目的定位，下面从功能覆盖、技术路线、交互方式、部署与扩展等维度，将本项目与若干代表性技术路线进行客观对比，如表 2-1 所示。对比对象均为公开的方法或工具类别，对比维度基于其公开资料与本项目代码实现，不含未经验证的量化结论。

表 2-1 本项目与同类技术路线对比

对比维度	传统结构化查重（如 MOSS / JPlag）	通用文本/AI 文本检测（如 DetectGPT 类）	直接整库喂入 LLM 的理解方式	本项目
主要对象	源代码词法/语法 token	自然语言文本为主	源代码文本	内核源码（函数级，C/Rust）
语义鲁棒性	对改名较鲁棒，对结构重写较弱	面向文本，不针对代码结构	依赖模型，易超上下文	AST 归一化+向量+分段，覆盖改名与重写

对比维度	传统结构化查重（如 MOSS / JPlag）	通用文本/AI 文本检测（如 DetectGPT 类）	直接整库喂入 LLM 的理解方式	本项目
是否需训练	免训练	多数免训练	免训练	查重与 AI 检测均免训练
抗幻觉机制	不涉及	不涉及	通常缺失	行号回源校验 + MCP 守卫 + LLM 复核
交互/产出	相似度分数/对比视图	判别分数	自由文本	分层 HTML 描述报告 + 查重对比报告
部署难度	较低	较低	低	中（依赖向量库与可选 GPU，提供降级）
可扩展性	规则相对固定	面向文本任务	受上下文限制	历史库可增量扩充、模型可替换

需要客观说明的是，MOSS、JPlag 等传统结构化查重工具在大规模代码查重场景中成熟稳定，DetectGPT 等方法在文本检测领域具有奠基意义；本项目并非否定上述工作，而是针对内核源码语义比对与抗幻觉评审这一具体场景，在技术路线上做了有针对性的组合与工程化。

2.5 本项目的优势、不足与创新点（概述）

综合上述分析，本项目的相对优势在于：以函数为粒度的 AST 归一化与向量召回，使其对改名与结构重写具有较好的鲁棒性；面向评审场景设计的抗幻觉机制，使描述与比较结论可回溯到真实代码行；三条链路共享同一数据底座，工程上较为统一。当前不足主要为：AI 生成代码检测依赖较大的打分模型与 GPU 资源，在无模型环境下仅能优雅跳过；描述报告端到端时延尚未形成系统化量化测试。创新点的详细论述见第五章 5.4 节，同类对比的结论亦在此处先行概述、后文不再展开重复。

第三章 系统框架设计

3.1 总体架构与设计原则

系统采用“通用脚手架层+领域专用层”的分层架构。通用脚手架层由开源智能体宿主 OpenCode 承载，提供 LLM 调用、上下文管理、工具协议与子代理等通用能力；领域专用层承载本项目的核心业务，包括查重算法、符号解析引擎、AI 检测集成层与抗幻觉报告生成。三条分析链路在同一份内核仓库之上并行展开：描述链路产出自底向上的理解树，查重链路以多层漏斗逐层收敛，AI 检测链路作为旁路对函数级代码打分。系统全景与分层关系分别如图 3-1、图 3-2 所示。

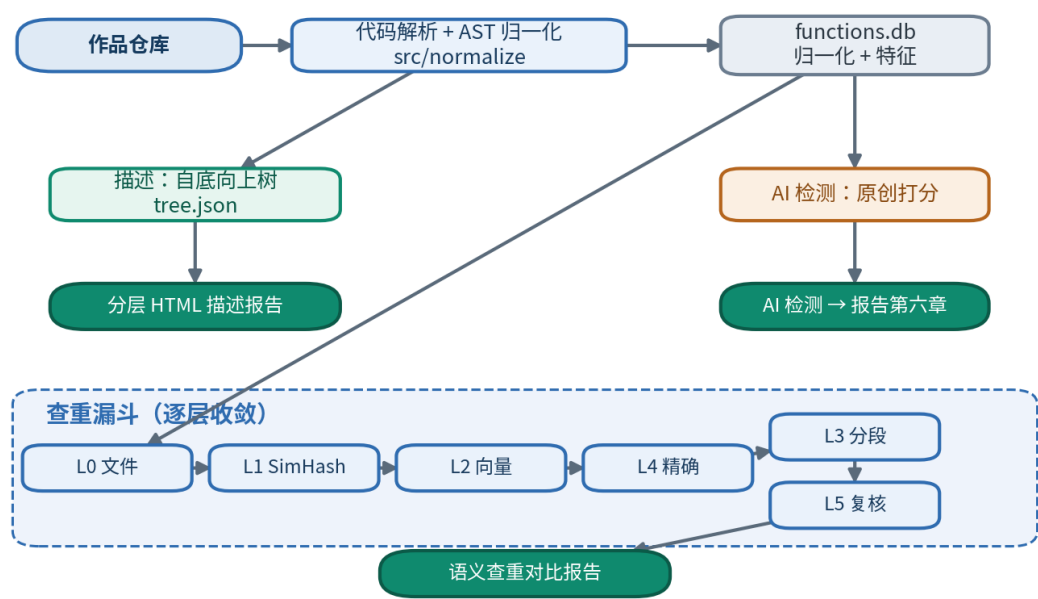


图 3-1 系统全景图：描述链路自底向上树+查重链路多层漏斗+AI 检测旁路



图 3-2 分层架构：通用脚手架层（OpenCode）承载通用能力，领域专用层承载核心业务

总体设计遵循四项原则：一是按需读取，模型不一次性加载整库，而是通过 MCP 工具按符号与行号范围读取，控制上下文规模；二是可核查，描述与比较结论尽量绑定到真实文件与行号，便于回源校验；三是可降级，关键外部依赖（向量库、GPU 打分模型、语言服务器）缺失时给出优雅降级路径而非直接失败；四是可扩展，历史库、嵌入模型与 LLM 均可替换或增量扩充。

需要说明的是，本系统为命令行与离线批处理形态，不含传统意义上的图形前端；本文所称“前端”指面向用户的产出形态，即分层 HTML 报告与命令行交互，“后端”指各分析链路的 Python 服务与引擎。

3.2 技术栈与运行环境

系统主体以 Python 实现，智能体宿主依赖 Node.js 运行的 OpenCode。模型方面，按赛题“优先国产/开源/免费模型”的导向，子系统理解与复核选用 DeepSeek 模型（经阿里云百炼 OpenAI 兼容接入点调用），语义嵌入选用 Salesforce 开源的 CodeT5+ 模型，AI 检测打分选用 bigcode 的 StarCoder2-3B 模型。完整技术栈如表 3-1 所示，各组件的来源归属在第九章统一说明。

表 3-1 系统技术栈

层次 / 组件	选型	作用
编程语言 / 运行时	Python ≥ 3.11（开发环境 3.12）、Node.js ≥ 18	分析主体逻辑；承载智能体宿主
智能体宿主	OpenCode	通用脚手架层：LLM 调用、上下文、工具协议、子代理
大语言模型	DeepSeek（deepseek-v4-flash，阿里云百炼接入）	子系统理解、嫌疑对复核、描述生成
工具协议	Model Context Protocol（MCP）	向 LLM 暴露读码 / 符号 / 搜索 / 对比工具
代码解析	tree-sitter（C/Rust）、rust-analyzer、clangd、ctags	多语言符号解析与 AST 归一化
语义嵌入	Salesforce/codet5p-110m-embedding	函数级语义向量
向量检索	Qdrant（可降级为本地 FAISS）	近邻召回（ANN）
指纹粗筛	SimHash（xxhash 加权）	L1 海明距离粗筛
AI 检测打分模型	bigcode/starcoder2-3b（可选 GPU）	log-rank / NPR 信号打分
存储	SQLite（functions.db / FTS5）、JSON、向量库	历史库、符号索引、流水线落盘
主要 Python 库	openai、gitpython、transformers、torch、numpy、scipy、pydantic、loguru、qdrant-client、python-gitlab	模型调用 / 解析 / 数值 / 数据校验 / 日志

3.3 模块划分与共享数据底座

三条链路在模块上相互独立，但共享同一套数据模型（src/models.py），其中 FunctionRecord 记录函数的仓库、文件、行号、模块标签、原始与归一化代码，Evidence 汇总各层证据（SimHash 距离、向量相似度、分段命中、精确与改名匹配行数、唯一字符串匹配、基线标记），SuspectPair 表示一对嫌疑函数及其最终分数、层级与匹配片段。共享数据模型如图 3-3 所示，三链路的模块划分如表 3-2 所

示。

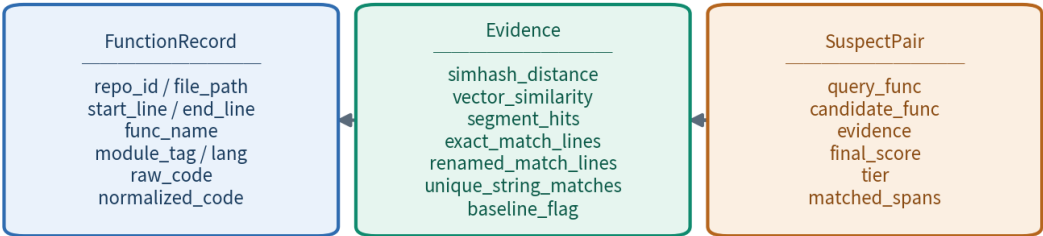


图 3-3 查重链路共享数据模型（src/models.py）

表 3-2 三条分析链路的模块划分

链路	模块	核心文件 / 目录	职责
描述报告	枚举与文件归类	parsers/code_parser.py	识别子系统并归类源文件
描述报告	符号索引与缓存	parsers/symbol_db.py、os_tools.py	符号定义 / 引用、FTS5 索引
描述报告	并行子系统分析	pipeline/tree_builder.py	自底向上构建理解树
描述报告	评判与渲染	reports/html_tree.py、html.py	VERDICT 评判与分层 HTML 报告
查重	解析归一化	normalize/	AST 切分与代码归一化
查重	L1 指纹粗筛	simhash/	SimHash 指纹与分段索引
查重	L2 向量召回	embed/	CodeT5+ 嵌入与 ANN 召回
查重	L3 分段验证	segment/	段级嵌入与匈牙利匹配
查重	L4 / L5 精确与复核	exact/、review/	行级比对与 LLM 多票复核
查重	元数据信号	metadata/	提交 / 字符串 / 基线辅助信号
查重	报告与编排	report/、pipeline/steps.py	对比报告与全流水线编排
AI 检测	特征提取	ai_detect/extract.py	桥接 normalize 解析为函数块
AI 检测	检测打分	ai_detect/runner.py、settings.py	log-rank / NPR 两阶段打分
AI 检测	集成（含第三方）	ai_detect/__main__.py、vendor/	并入报告；上游 DetectCodeGPT 代码

3.4 作品描述报告链路

描述链路的主管道为“定位仓库→构建仓库事实档案→自底向上构树→输出终端树与 HTML 报告”，如图 3-4 所示。系统首先对目标仓库做枚举与文件归类，随后由 LLM 在 MCP 工具辅助下逐子系统理解，最终汇总为顶层评判（VERDICT）并渲染为分层 HTML 报告。

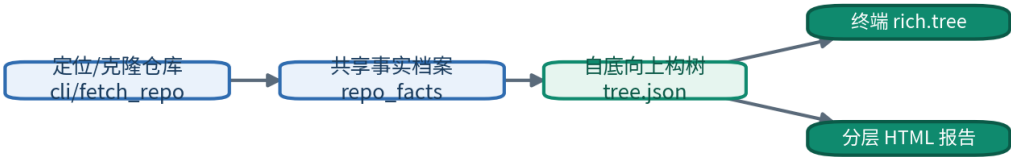


图 3-4 描述链路主管道：定位→事实档案→构树→终端树与 HTML 报告

3.4.1 多语言符号解析引擎

符号解析采用三档降级链以兼顾精度与鲁棒性：路径 A 使用 rust-analyzer 解析 Rust 工程，路径 B 使用 clangd 解析 C/C++（依赖 compile_commands.json），二者均为语言服务器协议（Language Server Protocol, LSP）实现；当 LSP 不可用或索引超时，降级为路径 C 的 universal-ctags 通用符号索引。引擎索引等待时间分别由 rust_analyzer_timeout（120 秒）与 clangd_timeout（60 秒）控制。三档降级链如图 3-5 所示。

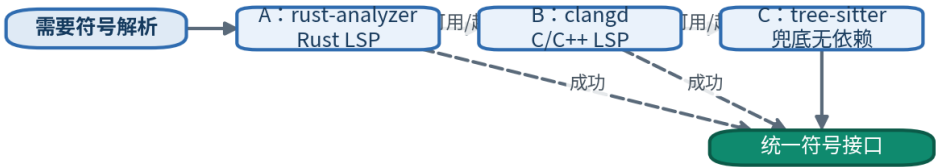


图 3-5 符号解析引擎 A / B / C 三档降级链

3.4.2 MCP 工具集与自底向上理解

系统通过 MCP 向 LLM 暴露一组只读分析工具，使模型以“按需读取”的方式探索仓库，而非一次性加载全部源码。主要工具如表 3-3 所示。自底向上的理解过程分为三阶段：枚举归类、子系统并发理解（SUBSYS）、顶层评判（VERDICT），如图 3-6 所示。为控制成本与避免失控空转，单会话探索类工具调用设有步数预算（max_steps=30）。

表 3-3 MCP 工具集与接口说明

工具名称	编号	功能说明
read_file	T1	读取指定文件源代码，返回带行号文本，可限定行号范围
find_symbol_definition	T2	查询符号定义位置
find_references	T3	查询符号引用位置

工具名称	编号	功能说明
expand_callees	T5b	按层展开被调函数，构造调用链
search_code	—	在仓库内进行代码检索
list_implemented_syscalls	T4	统计内核已实现的系统调用覆盖率
compare_with_reference_os	T6	与参考 OS 进行对比
analyze_subtree	T8	对子树/子系统做聚合分析

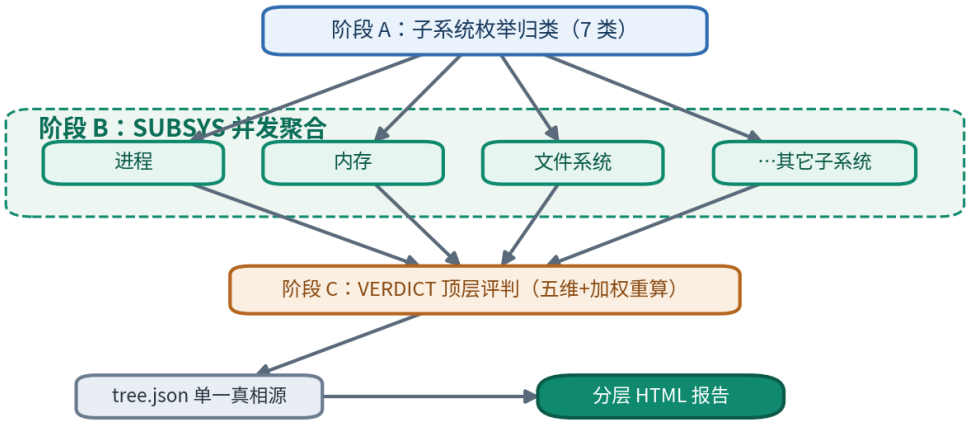


图 3-6 自底向上三阶段：枚举归类→SUBSYS 并发理解→VERDICT 顶层评判

3.5 历史作品查重链路

查重链路以多层漏斗逐层收敛，自上而下设 L0 文件指纹、L1 SimHash 粗筛、L2 向量召回、L3 分段验证、L4 精确比对、L5 LLM 复核六层，并辅以元数据信号与基线扣除。需要区分的是“概念层级”与“执行顺序”：流水线实际执行顺序为 ingest→fastpath（L0）→recall（在线归一化+L1 SimHash+L2 向量召回）→exact（L4）→segment（L3）→metadata→ai_detect→report，如图 3-7 所示。



图 3-7 查重全流水线七步串联（含 AI 检测旁路）

3.5.1 历史库构建

历史库通过一键建库流程构建：ingest 克隆历史仓库并抽取元数据，normalize 完成 AST 切分与归一化，embed 与 simhash 分别建立向量库与指纹索引，结果写入 data/db，如图 3-8 所示。历史仓库清单

维护于 config/repos.yaml，当前收录 167 个历届作品仓库，覆盖 2021—2025 年，其中含特等奖 1 项、一等奖 26 项（另含一等奖 MVP 1 项），其余未标注奖项；设计上目标库规模约 200 个仓库、约 22 万个函数（以仓库图示标注为准，实际以建库时配置为准）。

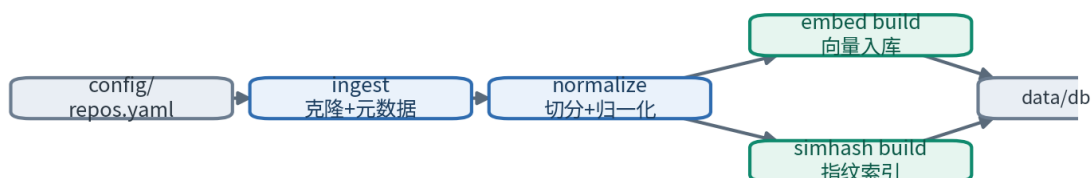


图 3-8 一键建库流程：ingest → normalize → embed → simhash

3.5.2 L1 SimHash 粗筛

L1 以加权 SimHash 为每个归一化函数生成 64 位指纹：对特征词以全库 IDF 加权，经 xxhash 映射后加权累加并符号化，得到指纹；指纹切为 4 段、每段 16 位建立倒排索引并允许比特松弛，从而以海明距离快速粗筛候选，如图 3-9 所示。

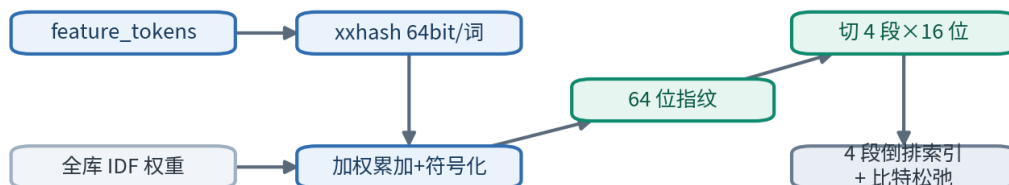


图 3-9 SimHash 指纹构造与 4×16 分段索引查表

3.5.3 L2 向量召回

L2 对新作品函数在线归一化后，使用 CodeT5+ 嵌入（滑窗 512、步长 384）生成语义向量，在 Qdrant 中按近邻检索召回每个查询函数的 top-k（默认 20）候选，如图 3-10 所示。向量召回对改名与结构重写较为鲁棒，是 SimHash 粗筛的语义兜底。

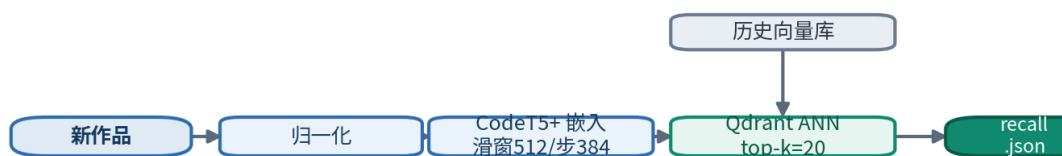


图 3-10 向量召回：归一化 → 嵌入 → Qdrant ANN top-k

3.5.4 L3 分段验证与 L4 精确比对

L3 将查询函数与候选函数各自切段、逐段嵌入，构造段对余弦相似度矩阵并以匈牙利算法求最优匹配，按覆盖率重打分以实现升降级，如图 3-11 所示。L4 在此基础上做精确行级与改名匹配，生成嫌疑对及其匹配片段。

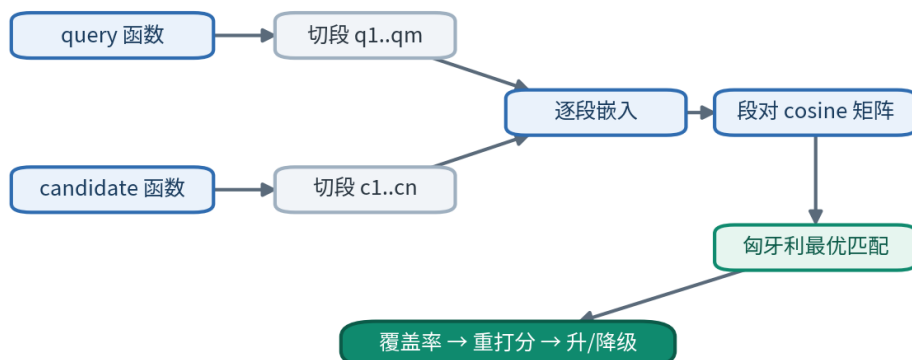


图 3-11 分段验证：切段 → 逐段嵌入 → 匈牙利最优匹配 → 覆盖率

3.5.5 元数据信号与基线扣除

元数据通道从提交历史与字符串等维度提供辅助信号，例如对早期即出现的完整实现、异常大提交、模糊提交信息等给出标记。为降低同源框架代码（如各类 rCore/uCore 衍生的公共代码）造成的误报，系统引入基线白底库扣除：仅当嫌疑对双侧均与同一基线函数的相似度高于阈值（`baseline_sim_threshold=0.85`）时才判为同源公共代码并予以扣除，如图 3-12 所示。

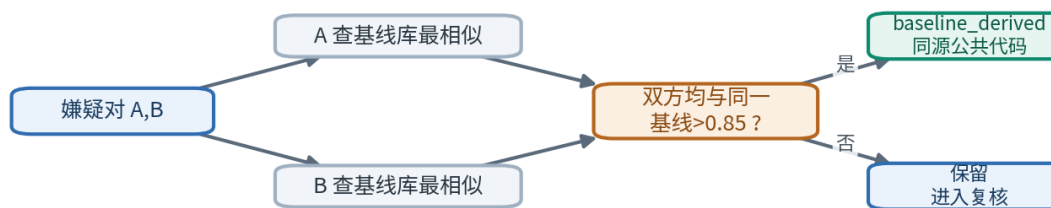


图 3-12 基线白底库扣除：仅双侧命中同一基线函数才扣除

3.5.6 L5 LLM 复核与报告

L5 对保留的嫌疑对构造嫌疑卡片交由 LLM 复核，采用三票多数表决（温度 0.3）以降低单次判断的随机性，并对结论中的行号引用做回源后置校验，如图 3-13 所示。最终由 report 模块生成语义查重对比报告。查重链路的关键阈值与配置集中列于表 3-4。

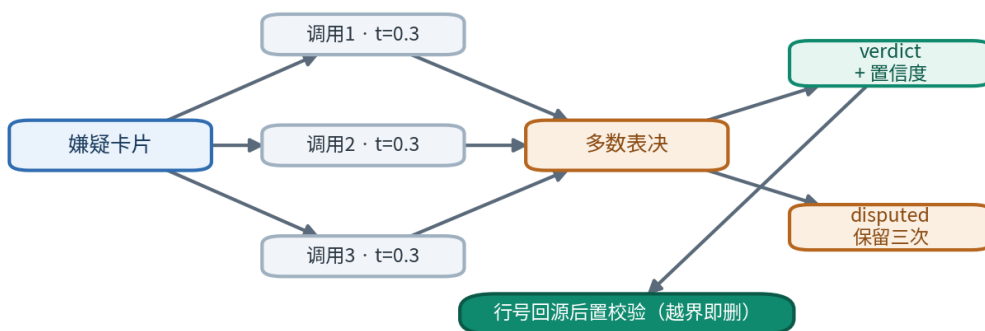


图 3-13 LLM 复核：三票多数表决+行号回源后置校验

表 3-4 查重与检测链路关键阈值与配置（取自 config/settings.yaml）

参数	取值	含义 / 作用
嵌入 max_length / window / stride	512 / 512 / 384	长函数滑动窗切分
batch_size	64	嵌入批大小（调优后，见第七章）
retrieval.top_k	20	每个查询函数召回候选数
baseline_sim_threshold	0.85	双侧同基线方可扣除
common_code_sim / repo	0.9 / 5	公共框架代码判定阈值
LLM votes / temperature	3 / 0.3	复核多数表决与温度
log_rank 阈值	0.31	AI 检测 Stage 1 判定线（针对 starcoder2-3b 标定）

参数	取值	含义 / 作用
NPR detect_score 阈值	0.20	AI 检测 Stage 2 辅助信号
min_loc	20	低于此有效行数判 Uncertain
max_steps	30	MCP 单会话探索步数预算

3.6 AI 生成代码检测链路

AI 生成代码检测采用免训练的两阶段信号方法。Stage 1 以 log-rank 为主信号做高置信判别：低于阈值判为疑似 AI 生成、高于阈值判为疑似真人；Stage 2 仅对落入灰区的函数施加扰动并计算 NPR 作为补判，从而在控制扰动成本的前提下提升边界样本的判别质量，如图 3-14 所示。该链路作为查重流水线的 ai_detect 步并入对比报告；在无 GPU 或未下载打分模型权重的环境下，该步骤优雅跳过并在报告中给出说明。检测方法的来源与归属见第九章。

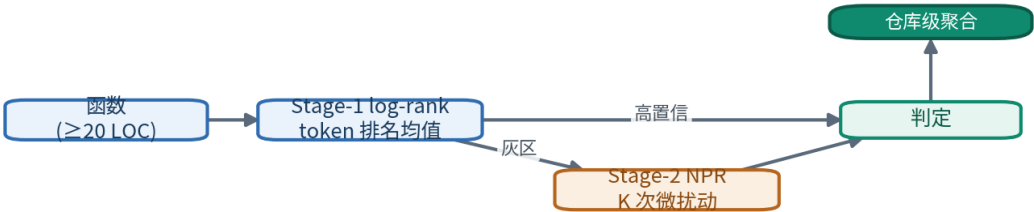


图 3-14 两阶段免训练检测：log-rank 主信号+NPR 灰区补判

3.7 接口设计、存储与运行流程

系统对外接口以命令行为主。描述链路通过 agent.py（或安装后的 oskernel-agent 命令）以 --url / --repo-path / --repo-id 指定目标仓库并产出 HTML 报告；查重链路以 python -m src.buildlib 一键建库、python -m src.pipeline --repo <路径或 URL> 产出对比报告；AI 检测既作为流水线步骤自动并入，也可经 python -m src.ai_detect 独立运行。对 LLM 与模型的调用接口经 OpenAI 兼容协议与 MCP 协议完成。存储方面，历史库与符号索引以 SQLite（functions.db、FTS5）与向量库形式持久化，流水线各步以 JSON 中间产物落盘，便于断点续跑（--resume-from）。

3.8 异常处理、降级与质量属性设计

在异常处理与降级方面，系统对关键外部依赖均设计了降级路径：向量库无 Docker 时降级为本地 FAISS；GPU 或打分模型缺失时 AI 检测步骤优雅跳过；LSP 不可用或索引超时时符号解析降级为 ctags。在稳定性方面，MCP 服务设有探索步数预算与“幻觉扩展守卫”，当模型反复查询由失败符号派生的变体名时累计计数，达到阈值即强制其停止探索、直接产出报告，避免在大子系统上空转至超时。在安

全性方面，分析工具均为只读，针对历史仓库中可能误提交的非法文件名（如含冒号的 `Windows Zone.Identifier` 文件）在克隆时以 `core.protectNTFS=false` 容错。在可维护性方面，三链路共享数据模型、配置集中于 `config/` 与 `settings.yaml`、第三方上游代码以 `vendor` 目录隔离存放，便于整目录升级。上述机制的具体问题背景见第七章。

第四章 开发计划

本章依据项目实际开发过程整理开发计划。计划的阶段划分与时间节点以代码仓库的提交历史为客观依据：仓库自 2026 年 4 月 19 日出现首次提交至 2026 年 6 月 30 日，共计约 120 次提交，按月分布为 4 月约 28 次、5 月约 20 次、6 月约 72 次，呈现“奠基—构建—集成收尾”的推进节奏。

4.1 阶段划分与时间进度

根据提交历史与模块出现顺序，项目开发划分为三个已完成阶段与一个后续阶段，如表 4-1 所示。各阶段并非严格串行，存在一定交叠，但主线清晰。

表 4-1 分阶段开发计划

阶段	时间	主要目标与任务	阶段性成果
阶段一 智能体与描述链路奠基	2026-04	搭建仓库分析 Agent；接入 MCP 工具；实现 C/Rust 符号解析、函数调用链路提取与代码核实	可对单个内核仓库产出初步结构化描述
阶段二 查重引擎构建	2026-05	实现归一化、SimHash、向量召回、精确比对、分段验证等多层漏斗与一键建库	可建立历史库并产出查重对比报告
阶段三 AI 检测、抗幻觉与评测收尾	2026-06	集成 AI 生成代码检测；引入基线扣除与行号回源校验；建立评测体系；批处理性能调优；完善报告与文档	查重查准率 1.0、查全率 0.975；整库嵌入显著提速
阶段四（后续） 优化与扩展	赛后计划	补齐描述链路端到端量化测试；评估替换嵌入 / LLM；扩充历史库与基线库	形成系统化性能基线与更完整评测

4.2 各阶段目标与阶段性成果

阶段一以打通“能读懂一份内核”为目标，重点在符号解析与按需读取，使模型能够在不超出上下文的前提下逐子系统理解代码。阶段二以打通“能比对历史库”为目标，重点在归一化与多层召回，使系统能在改名与重写等变形下稳定召回相似实现。阶段三以“可信与可用”为目标，重点在抗幻觉机制、AI 检测集成与评测闭环，并通过工程调优改善吞吐与稳定性。各阶段成果均落盘为可运行代码与可复现的中间产物。

4.3 后续优化计划

后续计划包括：其一，针对描述链路建立端到端量化测试，形成单仓库分析时延与 token 成本的系统化基线；其二，开展嵌入模型与 LLM 的可替换性评估，验证在更换模型后阈值的重标定流程；其三，扩充历史库与基线公共代码库，提升对新框架变体的覆盖；其四，完善对比报告的可视化与可读性。上述计划属设计与规划内容，本文不将其表述为已完成功能。

第五章 比赛过程中的重要进展

本章记录项目从初始方案到当前版本的重要进展，以体现完整、真实的开发过程。进展节点以提交历史与仓库内评测记录为依据，关键里程碑如表 5-1 所示。

表 5-1 重要进展里程碑（依据提交历史与评测记录）

时间	重要进展
2026-04-19	提交代码仓库分析 Agent 系统初始版本
2026-04-22 / 23	实现函数调用链路自动提取、结构体定义查询与代码核实；新增 Rust 语言解析器
2026-05	查重多层漏斗成形：归一化、SimHash 粗筛、向量召回、精确比对、分段验证
2026-06-13	查重评测基线：查准率 0.995、查全率 0.97（误报 1），整库嵌入耗时约 187 秒
2026-06-24	评测复测：查准率 1.0、查全率 0.975（误报 0），整库嵌入耗时约 6.4 秒
2026-06-29	AI 检测打分模型定为 starcoder2-3b 并修复检测崩溃；修复报告渲染标签不平衡
2026-06-30	报告新增待复核清单与 AI 检测章节；修复 facts 编码崩溃，精简报告与 README

5.1 核心模块的迭代过程

描述链路自初始的“调用链提取+代码核实”逐步扩展为“三档符号解析降级链+自底向上三阶段理解”，并补充了顶层评判与抗幻觉后置校验。查重链路自单一相似度比较逐步演化为多层漏斗，并在评测驱动下补入基线公共代码扣除以抑制同源框架代码误报。AI 检测链路则从直接套用工具默认阈值，调整为针对所选打分模型重新标定阈值的两阶段方案。

5.2 代码结构与工程的重要变化

工程结构上的重要变化包括：将查重引擎按 ingest / normalize / simhash / embed / segment / exact / metadata / report 拆分为职责清晰的模块，并以 pipeline/steps.py 统一编排；将 AI 检测的第三方上游代码以 vendor 目录原样隔离存放，集成层仅复用其检测链并桥接本仓库的 tree-sitter 解析，从而规避依赖冲突；将 MCP 服务加入步数预算与幻觉守卫以提升稳定性。这些变化使系统在功能扩展的同时保持了可维护性。

5.3 模型、接口与部署方面的进展

模型方面，子系统理解与复核确定使用 DeepSeek 模型（阿里云百炼接入），语义嵌入确定使用 CodeT5+，AI 检测打分模型在迭代中确定为 StarCoder2-3B（相较更大的 7B 级模型更轻、更适配 8GB 显存）。接口方面，描述、查重、AI 检测三类命令行入口与流水线断点续跑能力趋于稳定。部署方面，系统提供一键安装脚本与 Docker 方式的向量库，并对无 Docker、无 GPU 等受限环境给出降级路径。

5.4 创新性分析

结合上述进展，本项目的创新点均与实际代码与系统功能相对应，概述如下，其工程实践要点如图 5-1 所示。

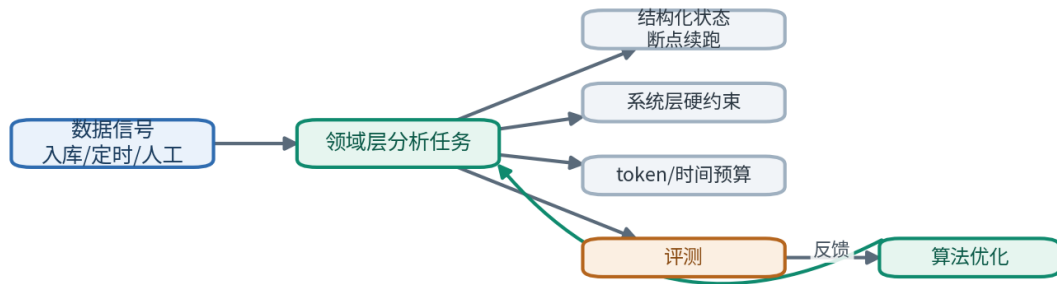


图 5-1 领域层 AI 设计实践：信号驱动+持久化状态+硬约束+成本控制+反馈闭环

第一，三链路同仓协同。描述、查重、AI 检测三条链路共享同一份内核仓库与同一套数据模型，在工程上统一服务于评审，避免了为不同任务重复解析与建模。

第二，语义鲁棒的多层漏斗查重。系统以 AST 归一化、向量召回与分段匈牙利匹配为核心，对改名（T2）与结构重写（T4）等变形提供语义兜底；评测数据显示，在 SimHash 单层召回偏低的变形类别上，向量层将召回提升至接近全召回（详见第六章）。

第三，面向评审的抗幻觉工程。系统在描述链路引入行号回源后置校验（对无法在真实代码区间内验证的句子整句删除并计数）、MCP 幻觉扩展守卫与探索步数预算，在查重链路引入精确行级比对与 LLM 多票复核，直接回应赛题“避免幻觉、精准无误”的评审要点。

第四，成本可控的免训练 AI 检测。检测采用两阶段信号，仅对灰区函数施加扰动，避免对全部函数做昂贵扰动，并在无模型环境下优雅跳过，兼顾质量与成本。

第五，基线公共代码扣除。针对教学型内核普遍衍生自公共框架的现实，系统以“双侧命中同一基线”为条件扣除同源公共代码，在评测中将误报降至 0（详见第六章）。

5.5 进展汇报幻灯片应体现的阶段性成果

为支撑后续作品进展汇报，建议在幻灯片中按阶段呈现以下可验证的阶段性成果：阶段一的单仓库描述能力与调用链提取；阶段二的多层漏斗与一键建库；阶段三的评测结果对比（查准率由 0.995 升至 1.0、误报由 1 降至 0、整库嵌入由约 187 秒降至约 6.4 秒）、AI 检测集成与抗幻觉机制。幻灯片的完整建议结构见附录 C。

第六章 系统测试情况

本章整理系统的测试情况。需要特别说明：本章涉及的查重评测数据均取自仓库内置的评测记录（tests/evaluation/history/ 下的评测报告），属可复现的真实结果；尚未形成系统化量化测试的部分（如描述链路端到端时延），均明确标注为待补充，不以推断数值替代。

6.1 测试环境

测试以仓库内置的评测脚本与单元测试为主，运行环境如表 6-1 所示。语义嵌入与 AI 检测打分均在 GPU 上运行；AI 检测打分模型 StarCoder2-3B 以 bf16 精度加载，显存占用约 6GB，可容纳于评测主机的 8GB 显存。

表 6-1 测试环境

项目	配置
处理器（CPU）	Intel Core i7-14650HX
显卡（GPU）	NVIDIA GeForce RTX 4060 Laptop（8GB 显存，用于嵌入与 AI 检测打分）
内存	32 GB
操作系统	Linux（开发亦支持 Windows，见 README）
语言运行时	Python 3.12；Node.js ≥ 18（OpenCode）
评测脚本	tests/evaluation/run.py、synthesize.py、transforms.py
单元测试	tests/ 下 14 个 pytest 模块
向量库	Qdrant（或本地 FAISS 降级）

6.2 功能测试

仓库在 tests/ 目录下提供 14 个 pytest 测试模块，覆盖查重与检测链路的主要模块，如表 6-2 所示。本文未在评审环境重新执行全部用例，建议在标准环境运行 pytest 以复核通过情况；下表反映的是测试覆盖范围而非本次执行的通过结论。

表 6-2 功能测试用例（仓库内置 pytest 覆盖）

测试模块	覆盖内容	备注
test_normalize	代码归一化与切分	解析预处理
test_simhash	SimHash 指纹与分段索引	L1
test_embed	向量嵌入与召回	L2
test_segment	分段匈牙利匹配	L3
test_exact	精确 / 改名行级比对	L4
test_review	LLM 复核投票	L5
test_metadata	提交 / 字符串 / 基线信号	辅助信号
test_ai_detect	AI 检测打分流程	AI 检测

测试模块	覆盖内容	备注
test_report / test_fastpath / test_ingest_* / test_toc / test_evaluation	报告、文件指纹、克隆与元数据、目录、评测	其余模块

6.3 查重链路评测：方法与结果

评测体系以合成的四类“已知克隆”为正样本注入历史库后做端到端检索，并配以负样本统计查准率与查全率，整体流程如图 6-1 所示。四类变换分别为：T1 原样复制、T2 系统性改名、T3 增删语句、T4 结构重写（如 if-else 与 match、for 与 while-let 互转），均基于 tree-sitter 保证语法合法。

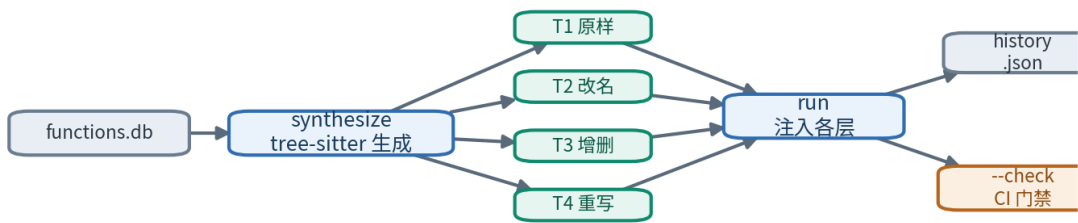


图 6-1 评测体系：合成四类克隆 → 注入各层 → 统计 → CI 门禁

最近一次评测（2026-06-24）的样本为 T1–T4 各 50、负样本 200。各类别在 L1（SimHash）、L2（向量）、级联与最终的召回率如表 6-3 所示。可以看出，SimHash 单层对改名（T2）与结构重写（T4）召回偏低（均为 0.44），而向量层将其提升至 0.98—1.0，体现了语义召回对文本变形的兜底作用。

表 6-3 查重评测各类召回率（2026-06-24，来源：仓库评测记录）

类别（变换）	样本数	L1 召回	L2 召回	级联召回	最终召回
T1 原样复制	50	1.00	1.00	1.00	1.00
T2 系统性改名	50	0.44	1.00	1.00	1.00
T3 增删语句	50	0.76	1.00	1.00	0.90
T4 结构重写	50	0.44	0.98	1.00	1.00
负样本	200	—	—	—	—

总体上，该次评测查准率为 1.0（真正例 195、假正例 0），查全率为 0.975（真正例 195、正样本 200）。

6.4 性能与精度的进展对比

将 2026-06-13 与 2026-06-24 两次评测对比，可观察到精度与性能的同步改善，如表 6-4 所示。其中整库嵌入耗时由约 187 秒降至约 6.4 秒，主要得益于将嵌入批大小由 16 调整为 64（详见第七章）；误报由 1 降至 0，主要得益于基线公共代码扣除的引入。

表 6-4 评测进展对比（来源：仓库评测记录）

指标	2026-06-13	2026-06-24	说明
查准率 precision	0.995	1.000	假正例 1 → 0
查全率 recall	0.970	0.975	正样本 200
整库嵌入耗时 embed_all	约 187.44 s	约 6.41 s	批大小 16 → 64
流水线耗时 pipeline	约 0.83 s	约 0.66 s	单次对比

6.5 接口、兼容性与异常测试

接口测试方面，描述、查重、AI 检测三类命令行入口与流水线断点续跑（--resume-from）在开发过程中持续使用并验证。兼容性方面，系统在 Linux 与 Windows 下均提供安装与解析方案（Windows 以导出 compile_commands.json 替代 Linux 的 bear），并对历史仓库中误提交的非法文件名以 core.protectNTFS=false 容错。异常测试方面，系统对向量库缺失、GPU/打分模型缺失、LSP 索引超时等情形均设计了降级路径，AI 检测在无模型时优雅跳过并在报告中说明；上述异常处理的具体背景见第七章。系统化的接口与兼容性测试用例尚不完整，标注为待补充。

6.6 功能完成情况

为避免将设计目标表述为已实现功能，下面以表 6-5 区分各功能的完成状态。状态分为“已实现”“部分实现”“计划/待补充”三类，并给出对应的代码依据。

表 6-5 功能完成情况

功能	状态	依据 / 说明
作品描述报告（自底向上 HTML）	已实现	agent.py 与 oskernel_agent 链路
多语言符号解析（Rust/C 三档降级）	已实现	engines/path_a、path_b、path_c
MCP 工具集（8 项只读工具）	已实现	cli/mcp_server.py
历史库一键构建	已实现	src.buildlib
多层漏斗查重与对比报告	已实现	src.pipeline、report/
基线公共代码扣除	已实现	metadata/baseline.py
LLM 复核（多票表决）	已实现	review/
行号回源后置校验（抗幻觉）	已实现	report/postcheck.py 等
AI 生成代码检测	已实现（缺 GPU/模型时优雅跳过）	src.ai_detect
查重评测体系	已实现	tests/evaluation
描述链路端到端量化测试	计划 / 待补充	暂无系统化数据

功能	状态	依据 / 说明
历史库达约 200 仓 / 22 万函数	部分实现	当前 config 收录 167 仓，目标规模待建库确认

6.7 功能分析

从功能关系看，三条链路共享归一化与数据模型：描述链路输出“理解”，查重链路输出“比较”，AI 检测链路输出“原创性信号”，后者并入查重报告形成统一产出。就赛题覆盖而言，描述链路对应“对历史作品进行描述”的任务，查重链路对应“新作品与历史作品比较”的任务，二者均产出 HTML 文档，满足“对人类友好的描述/比较文档”的要求。在运行演示中可直接体现的功能点包括：单仓库描述报告的生成、查重对比报告中嫌疑对与匹配片段的展示、AI 检测章节与待复核清单。

6.8 性能分析

基于可验证的评测记录，查重链路在最近一次评测中整库嵌入耗时约 6.4 秒、单次对比流水线耗时约 0.66 秒，查准率 1.0、查全率 0.975，表明在当前样本规模下具备较好的效率与精度。资源占用方面，语义嵌入与 AI 检测打分依赖 GPU，AI 检测打分模型显存需求约 6GB（bf16）。需要客观指出的是，描述链路的端到端时延与 token 成本当前阶段暂未形成系统化量化测试，后续需补充；本文不对其给出具体数值。

6.9 测试结论

综合现有测试与评测记录，可得出如下结论：查重链路在合成的四类已知克隆与负样本上达到查准率 1.0、查全率 0.975，并在迭代中实现误报下降与嵌入提速；功能层面，描述、查重、AI 检测三条链路的核心功能均已实现，且对受限环境提供降级；尚需补充的工作包括描述链路的端到端量化测试与系统化的接口/兼容性测试。上述结论均以仓库内代码与评测记录为依据。

第七章 遇到的主要问题和解决方法

本章按“问题现象—原因分析—解决方案—结果验证”的结构，总结研发过程中遇到的主要工程问题。所列问题均来自代码注释、配置说明与提交记录，解决方案与代码实现一致。

7.1 环境配置与依赖冲突

7.1.1 第三方解析器依赖与本仓库 tree-sitter 版本冲突

问题现象：直接引入 AI 检测上游项目的特征提取模块会引解析器依赖冲突，无法与本仓库共存。

原因分析：上游 extractor.py 依赖 tree-sitter-languages（绑定 tree-sitter≈0.21），而本仓库锁定 tree-sitter==0.25.2；同时 CodeT5+ 的 trust_remote_code 模型仅兼容 transformers 4.x，不能升级到 5.x。

解决方案：在集成层 ai_detect/extract.py 不导入上游 extractor.py，改用本仓库 normalize 的 tree-sitter 解析器，将函数桥接为上游所需的数据结构；并在依赖文件中固定 transformers 版本区间。

结果验证：AI 检测链路在不安装 tree-sitter-languages 的前提下正常运行，且上游代码以 vendor 目录原样保留，便于整目录升级。

7.1.2 Windows 环境下历史仓库检出失败

问题现象：克隆部分历史仓库时，在 Windows 上检出失败。

原因分析：个别历史仓库误提交了含冒号的 Windows Zone.Identifier 等非法文件名，触发 NTFS 文件名保护。

解决方案：在克隆命令中加入 core.protectNTFS=false 以容忍非法文件名，并采用浅克隆控制体积。

结果验证：相关仓库可正常克隆与建库，跨平台流程恢复一致。

7.2 模型加载与推理

7.2.1 AI 检测阈值与打分模型失配

问题现象：沿用检测工具的默认阈值时，几乎所有代码都被判为 AI 生成。

原因分析：log-rank 等信号的判定阈值与打分模型强相关，工具原默认值（如 1.5/3.0）并不适配本系统选定的 StarCoder2-3B。

解决方案：针对 StarCoder2-3B 重新标定阈值（log_rank 判定线取 0.31，NPR 辅助阈值取 0.20），并采用两阶段判别，仅对灰区函数施加扰动。

结果验证：AI 检测对真人与疑似 AI 代码的区分趋于合理；配置中明确注明“更换打分模型必须重新标定阈值”，避免误用。

7.2.2 语义嵌入吞吐瓶颈

问题现象：整库嵌入耗时偏长，GPU 利用率低。

原因分析：嵌入批大小过小（原为 16），GPU 利用率仅约 5%，未能发挥并行能力。

解决方案：将嵌入批大小调整为 64，提高单批窗口数以提升 GPU 利用率。

结果验证：GPU 利用率提升至约 40%，整库嵌入耗时由约 187 秒降至约 6.4 秒（见第六章表 6-4），精度未见下降。

7.3 运行稳定性

7.3.1 智能体探索失控与幻觉扩展

问题现象：在内存管理、设备驱动等大子系统上，推理模型可能持续空转直至超时。

原因分析：原探索步数预算过大（200），且模型可能反复查询由失败符号派生的“幻觉”变体名，陷入无效循环。

解决方案：将单会话探索步数预算下调为 30（write_report 等写盘动作不计步），并加入幻觉扩展守卫：当检测到查询符号是已失败符号的后缀变体时累计计数，达到阈值即强制模型停止探索、直接产出报告。

结果验证：大子系统分析不再失控空转，单会话在预算内收敛产出报告。

7.3.2 报告渲染与数据编码崩溃

问题现象：报告渲染偶发标签不平衡导致显示异常；事实档案在特定编码下崩溃。

原因分析：HTML 片段中存在未闭合标签；事实档案处理时未统一字符编码。

解决方案：为报告渲染加入标签平衡处理，统一事实档案的编码处理路径（见提交记录 2026-06-29 与 2026-06-30）。

结果验证：报告渲染异常与编码崩溃问题消除，输出稳定。

7.4 算法精度与数据处理

7.4.1 SimHash 对改名与结构重写召回不足

问题现象：对系统性改名（T2）与结构重写（T4）样本，SimHash 单层召回偏低（约 0.44）。

原因分析：SimHash 基于词法特征，对标识符改名与控制结构改写较为敏感。

解决方案：在 SimHash 粗筛之后串联向量召回与分段验证，以语义相似度兜底文本变形。

结果验证：向量层将 T2/T4 召回提升至 0.98—1.0，级联召回达 1.0（见第六章表 6-3）。

7.4.2 同源公共代码导致误报

问题现象：教学型内核普遍衍生自公共框架，导致同源公共代码被误判为抄袭。

原因分析：公共框架代码在多个仓库高度相似，仅凭相似度无法区分“公共”与“抄袭”。

解决方案：引入基线白底库扣除：仅当嫌疑对双侧均与同一基线函数相似度高于 0.85 时，才判为同源公共代码并扣除。

结果验证：评测假正例由 1 降至 0，查准率由 0.995 升至 1.0（见第六章表 6-4）。

7.5 数据可信与抗幻觉

7.5.1 LLM 生成的行号引用不可靠

问题现象：模型在描述中可能给出并不存在于真实代码区间的“文件:行号”引用。

原因分析：长上下文分析中模型存在幻觉，可能编造行号。

解决方案：构造报告时记录喂给模型的全部真实“文件:行号”区间作为白名单，对结论逐句抽取行号引用并回源校验，无法验证的句子整句删除并在报告末尾统计删除条数；在调用写报告前另设文件存在性校验。

结果验证：报告中保留的行号引用均可回源到真实代码，直接支撑赛题“避免幻觉、精准无误”的评审要点。

第八章 分工和协作

本章整理团队分工与协作方式。本项目由杭州电子科技大学“海底小纵队”团队完成，团队成员为牛一强、王佳鑫、王一凡，指导教师为曾英佩。依据代码仓库的 Git 提交记录，三名成员的提交占比约为 47%、33%、19%。下文按系统模块给出团队分工，部分成员同时承担多个角色。

8.1 团队分工

结合系统的模块构成，团队分工如表 8-1 所示。

表 8-1 团队分工

角色 / 职责	负责人	主要工作内容
项目负责人与总体架构	牛一强	技术选型、架构设计、进度统筹、分层与降级策略
描述链路 with 符号解析	牛一强	oskernel_agent、MCP 工具、引擎 A/B/C、自底向上构树
查重引擎与算法	王佳鑫	归一化、SimHash、向量召回、分段、精确比对
数据与历史库	王佳鑫	repos.yaml、历史库构建、基线公共代码库
AI 检测与模型集成	王一凡	ai_detect、阈值标定、上游 DetectCodeGPT 集成
测试与评测	王一凡	pytest 用例、评测体系、CI 门禁
部署与运维	牛一强	setup.sh、Docker 向量库、受限环境降级
文档与汇报材料	全体成员	技术文档、进展汇报 PPT、演示视频

说明：上表为团队的实际分工安排，部分成员同时承担多个角色，文档与汇报材料由全体成员共同完成。

8.2 协作方式与任务推进

团队以代码仓库为中心开展协作：通过分支与提交推进功能开发，以评测脚本与单元测试驱动质量改进，以仓库内的开发记录与交接说明（如 handoff.md）实现阶段性交接。仓库根目录的约定文件（CLAUDE.md）明确了提交、文档存放与协作约定，例如要求文档统一存放于 work/ 目录、提交信息不含无关署名等，保证协作过程的一致与可追溯。

8.3 代码托管与版本管理

项目代码托管于全国赛指定平台 gitlab.eduxiji.net（仓库地址见封面与第九章）。版本管理上，项目采用 Git 进行多次提交（共约 120 次，时间跨度为 2026 年 4 月至 6 月），并配置了 GitLab CI（.gitlab-ci.yml）与评测门禁脚本，以在合并前对查重评测指标进行校验。完整、真实的开发过程可由提交历史回溯。

第九章 提交仓库目录和文件描述

本章说明提交仓库的目录结构与文件作用，并以独立小节明确区分本团队开发内容与非本团队来源内容，严格避免将第三方代码、模型或数据表述为完全原创。

9.1 仓库总体结构

仓库以 Python 为主体，核心分析代码位于 `src/`，配置位于 `config/`，历史库与数据位于 `data/` 等目录，测试与评测位于 `tests/`。系统为命令行与离线批处理形态，不含独立的图形前端工程；面向用户的产出为分层 HTML 报告。主要目录与文件如表 9-1 所示。

表 9-1 提交仓库目录与文件说明

文件 / 目录	作用	本团队开发	备注
README.md	部署与使用说明	是	三链路命令速览
agent.py	描述链路根入口	是	转发 <code>oskernel_agent.cli.agent</code>
pyproject.toml、requirements.txt	打包与依赖声明	是	声明第三方库
config.toml、config/	运行配置与历史库清单	是	<code>settings.yaml</code> 、 <code>repos.yaml</code> 等
setup.sh、setup_opencode.py	一键安装与 Agent 注册	是	调用第三方工具安装
docker-compose.yml	向量库（Qdrant）服务编排	是	引用第三方镜像
src/oskernel_agent/	描述报告链路	是	MCP、引擎、构树、报告、抗幻觉
src/（normalize、simhash、embed、segment、exact、review、metadata、fastpath、report、pipeline、buildlib）	查重链路（多层漏斗）	是	L0-L5 与编排
src/ai_detect/（extract、runner、settings）	AI 检测集成层	是	桥接解析与打分
src/ai_detect/vendor/ai_code_detector/	AI 检测上游算法	否（第三方）	DetectCodeGPT 原样拷贝
src/models.py	共享数据模型	是	FunctionRecord / Evidence / SuspectPair
analysis/	子系统模块拆分文档	是	领域知识与分析产物
data/、history_db/、reference_db/	历史库与参考数据	部分	含外部克隆的历史作品（第三方）
tests/	单元测试与评测体系	是	pytest 与 evaluation
scripts/、tools/	辅助脚本	是	爬取、建库、校验

9.2 非本团队来源说明

系统在开源框架、第三方模型与公开算法的基础上完成集成与二次开发。下表 9-2 逐项列出主要的非本团队来源资源、类型、来源/许可情况，以及团队在其上完成的工作。团队对上述资源的使用均以集成、调用、桥接或自实现为主，未将其表述为完全自主原创；正式提交时应遵循各资源相应的开

源许可与引用要求。

表 9-2 非本团队来源说明

资源	类型	来源 / 许可	团队使用 / 二次开发
OpenCode	智能体宿主框架	开源项目	作为通用脚手架层，注册自定义 Agent 与 MCP 工具
DeepSeek（deepseek-v4-flash）	大语言模型 / API	第三方模型，阿里云百炼接入	调用做子系统理解与复核（密钥自备）
codet5p-110m-embedding	嵌入模型	Salesforce，开源	用于函数级语义向量
starcoder2-3b	打分模型	BigCode，开源	用于 log-rank / NPR 打分；重新标定阈值
DetectCodeGPT（ai-code-detector）	AI 检测算法 / 代码	第三方上游，vendor 原样拷贝	仅复用检测链，桥接本仓库解析
tree-sitter（C/Rust 语法）	解析库	开源	归一化与特征提取
rust-analyzer / clangd / ctags / bear	语言服务器 / 索引工具	开源	三档符号解析降级链
Qdrant / FAISS	向量数据库	开源	近邻召回（FAISS 为降级）
SimHash	算法	公开算法（Charikar, 2002）	自实现加权 SimHash 指纹
历史决赛作品仓库	外部数据 / 作品	各参赛队，eduxiji 平台托管	仅作历史比对库，版权归原作者
transformers、torch、openai 等	开源 Python 库	PyPI	作为依赖调用

此外，本文部分图表沿用并整理自项目旧技术文档中的架构图与流程图（由项目自有脚本生成），术语与系统描述在旧文档基础上重新整理；赛题背景文字依据华中科技大学发布的赛题说明。上述来源在第十一部分参考资料中统一列出。

第十章 比赛收获

本章从技术能力、工程实践、团队协作、问题解决、文档与测试规范、项目管理等方面，客观总结项目过程中的收获。

10.1 技术能力

通过本项目，团队较为系统地实践了大语言模型智能体的工程方法，包括以 MCP 协议向模型暴露受控工具、以按需读取控制上下文规模、以检索增强与技能化提示组织领域知识。同时，团队在代码语义表示（CodeT5+ 嵌入）、近邻检索（Qdrant/FAISS）、指纹粗筛（加权 SimHash）与免训练 AI 生成代码检测（log-rank/NPR）等方向积累了可迁移的技术经验。

10.2 工程实践

在工程层面，团队形成了若干可复用的实践：以分层架构区隔通用脚手架与领域逻辑；为关键外部依赖设计降级路径以提升鲁棒性；以 vendor 目录隔离第三方上游代码、规避依赖冲突；以评测脚本与持续集成门禁驱动质量改进。性能调优（嵌入批大小）与稳定性加固（步数预算与幻觉守卫）的经验，体现了以数据与现象为依据解决工程问题的方法。

10.3 团队协作与项目管理

团队以代码仓库为中心组织协作，通过约定文件统一提交与文档规范，通过交接说明实现阶段性衔接，并以多次提交记录完整保留了真实的开发过程。这一过程使团队体会到清晰的分工约定、可追溯的版本管理与持续的测试反馈，对项目按期推进具有重要作用。

10.4 问题解决与文档/测试规范

面对阈值失配、依赖冲突、性能瓶颈、模型幻觉等典型问题，团队按“现象—原因—方案—验证”的方法逐项处置，并将结论沉淀到配置说明与代码注释中。围绕赛题“避免幻觉、精准无误”的要求，团队建立了行号回源校验等可核查机制，并在文档撰写中坚持“以代码与评测记录为依据、区分已实现与计划、标注待补充”的规范。上述收获将持续应用于后续优化与扩展工作。

参考资料

本文涉及的赛题、方法、模型与工具的来源如下。学术方法引用以可公开检索的版本为准；其中 DetectCodeGPT 上游集成说明见项目 vendor 目录。

- [1] 2026 年全国大学生计算机系统能力大赛操作系统设计赛 OS 功能挑战赛道赛题说明（华中科技大学，命题人：邵志远）。
- [2] E. Mitchell, et al. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. 2023. arXiv:2301.11305.
- [3] J. Su, et al. DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text. 2023. arXiv:2306.05540.（NPR 信号来源）
- [4] Y. Shi, et al. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers（DetectCodeGPT）. 2024. arXiv:2401.06461.
- [5] M. S. Charikar. Similarity Estimation Techniques from Rounding Algorithms（SimHash）. ACM STOC, 2002.
- [6] Y. Wang, et al. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. 2023. arXiv:2305.07922.（模型 Salesforce/codet5p-110m-embedding）
- [7] Model Context Protocol（MCP）官方文档，Anthropic。
- [8] Qdrant 向量数据库官方文档；FAISS 项目文档。
- [9] tree-sitter 官方文档；rust-analyzer、clangd（LLVM）、universal-ctags 项目文档。
- [10] DeepSeek 模型说明；阿里云百炼 OpenAI 兼容接入文档。
- [11] BigCode StarCoder2 模型卡（bigcode/starcoder2-3b, HuggingFace）。
- [12] OpenCode 智能体宿主项目说明。
- [13] 项目旧技术文档（团队自有，本文据其结构、术语与图表重构）。